Homework 2: Two-way Min-cut Partitioning

Name: 陸明德

Student ID: 111062693

How to compile and execute program ?

How to Compile

Because this program is using boost C++ library, please make sure that you compile the program with ic21, ic22, ic51, and ic55 server.
In "HW2/src/", enter the following command to compile the program.

$ make

An executable file "hw2" will be generated in "HW2/bin/".
If you want to remove it, please enter the following command.

$ make clean

How to Run

Usage:

$ ./hw2 <text_file> <out_file>

Example: in "HW2/bin/", enter the following command:

$ ./hw2 ../testcase/public1.txt ../output/public1.out

What's the final cut size and the runtime of each testcase ?

Testcase	Cut Size	Runtime
public1	193	0.03
public2	3666	1.63
public3	7092	50.23
public4	2265	1.19
public5	1669	15.77
public6	10281	233.59

What's the details of your algorithm ?

Fiduccia-Mattheyses (FM) Algorithm is used in this program.
Implement the algorithm based on the class slides. However, the data structure is implemented by myself and some optimization is done.

1. Die structure

```
struct die{
    string techs;
    long double util;
    long double max_area;
    long double area;
};
```

1. techs : the technology of the die
2. util : the max utilization of the die
3. max_area : the max area of the die
4. area : the current area of the die

2. Cell structure

```
struct cell{
    string cellName;
    string libCellName;
    vector<string> connectNets;
    die* fromBlock, *toBlock;
    long long gain;
    bool locked;
};
```

1. cellName : the name of the cell
2. libCellName : the library cell name of the cell
3. connectNets : the nets that the cell is connected to (net name)
4. fromBlock : the die that the cell is currently in (pointer to die)
5. toBlock : the die that the cell will be moved to (pointer to die)
6. gain : the gain of the cell
7. locked : whether the cell is locked or not

Cell is stored in a unordered_map, which is a hash table. The key is the cell name, and the value is the cell itself.

3. Other data structure

```
unordered_map<string, vector<cell*>> nets;
unordered_map<string, unordered_map<string, long double>> techs;
map<long long, list<cell*>> buckets;
```

1. for nets, the key is the net name, and the value is a vector of cells that are connected to the net.
2. for techs, the key is the technology name, and the value is a hash table. The key of the hash table is the cell name, and the value is the area of the cell in the technology.

3. for buckets, the key is the gain, and the value is a list of cells that have the same gain.

Fiduccia-Mattheyses Algorithm

Iteration part

```
do{
    for (auto it = buckets.rbegin(); it != buckets.rend(); it++) {
        for(auto it2: it->second){
            if(util_condition(it2->toBlock, techs[it2->toBlock->techs][it2->libCellName])){

                update_gain(it2, nets, techs, buckets);

                buckets[it2->gain].remove(it2);
                if(buckets[it2->gain].empty()){
                    buckets.erase(it2->gain);
                }

                cellStk.push(it2);
                gain_sum += it2->gain;
                step++;
                if(gain_sum >= max_gain_sum){
                    max_gain_sum = gain_sum;
                    max_step = step;
                }

                goto next;
            }
        }
    }
    // means no cell can be moved
    break;
    next:
    if(buckets.empty()){
        break;
    }
    if((clock() - start_time) / CLOCKS_PER_SEC >= 280){
        break;
    }

}while(gain_sum > 0);

// recover to max gain sum state using stack
max_step = step - max_step;
for(long long i = 0; i < max_step; i++){
    cell* c = cellStk.top();
    cellStk.pop();
    c->fromBlock->area -= techs[c->fromBlock->techs][c->libCellName];
    c->toBlock->area += techs[c->toBlock->techs][c->libCellName];
    swap(c->fromBlock, c->toBlock);
}
```

1. First extract the cell with the highest gain from the bucket.
2. If the cell is condition satisfied, update the gain of the cell and move the cell to the other die.
3. Update the gain of the cell that is connected to the cell that is moved.
4. If the cell is not condition satisfied, continue checking the next cell.
5. If there is no cell can be moved, break the loop.
6. Or if the runtime is over 280 seconds, break the loop. (I set the time limit to 280 seconds, cause the limit of the runtime is 300 seconds.)
7. Or if the gain sum smaller or equal to zero, break the loop.
8. Otherwise, continue the loop.

Here I use a stack to store the cells that are moved. In the end, I will recover the die to the state that has the highest gain sum.

Beside, following code is check every iteration whether the solution have more gain to improve. If so, reset the buckets and continue the loop and using the state that has the highest gain sum as the initial state. Otherwise, break the loop.

```
while(true){

    // previous code (Iteration part)

    if(step <= 1){
        break;
    }

    if((clock() - start_time) / CLOCKS_PER_SEC >= 280){
        break;
    }

    buckets.clear();
    for(auto it = cells.begin(); it != cells.end(); it++){
        long long gain = Gn(&(it->second), nets);
        buckets[gain].push_back(&it->second);
        it->second.gain = gain;
        it->second.locked = false;
    }
}
```

What tricks did you use to enhance your solution quality?

Initial Partitioning

The initial partitioning is the most important part of the algorithm.

1. Cause the initial partitioning will determine the initial cut size, and the cut size will affect the runtime of the algorithm.
2. Beside, since the initial cut size is the lower bound of the final cut size. The better the initial cut size is, the better the final cut size will be.

I've tried several ways to do the initial partitioning, and the best way I found is to use the following formula will give the best result.

```
for(auto &it : cellNames){
    if(((techs[dA.techs][cells[it].libCellName])  / dA.util ) <= ((techs[dB.techs]
[cells[it].libCellName]) / dB.util)){
        if(util_condition(&dA, techs[dA.techs][cells[it].libCellName])){
            cells[it].fromBlock = &dA;
            cells[it].toBlock = &dB;
            dA.area += techs[dA.techs][cells[it].libCellName];
        }
        else{
            cells[it].fromBlock = &dB;
            cells[it].toBlock = &dA;
            dB.area += techs[dB.techs][cells[it].libCellName];
        }
    }
    else{
        if(util_condition(&dB, techs[dB.techs][cells[it].libCellName])){
            cells[it].fromBlock = &dB;
            cells[it].toBlock = &dA;
            dB.area += techs[dB.techs][cells[it].libCellName];
        }
        else{
            cells[it].fromBlock = &dA;
            cells[it].toBlock = &dB;
            dA.area += techs[dA.techs][cells[it].libCellName];
        }
    }
}
```

1. The formula is to compare the area of the cell in the technology of the die, and divide it by the max utilization of the die. The die with the smaller result will be the die that the cell will be assigned to. However, if the die is already full, the cell will be assigned to the other die.\
2. **Here is a tricky part. The order of the cell is important. It's order must be same as the order of the cell in the input file. Otherwise, the result will be different.**

Furthermore, I do some optimization to the initial partitioning.

1. Since the goal is to minimize the initial cut size, I caculate of the gain of each cell.
2. sort the cells by the gain.
3. Then, I move the cells with the highest gain first.
4. If the cell is condition satisfied and gain is larger than zero, I move the cell to the other die. Otherwise, continue checking the next cell.

```
priority_queue<pair<long long, cell*>> pq;

for(auto &it : cells){
    pq.push({Gn(&it.second, nets), &it.second});
```

```
}

while(!pq.empty()){
    auto it = pq.top();
    pq.pop();
    if(it.first > 0){
        if(util_condition(it.second->toBlock , techs[it.second->toBlock->techs]
[it.second->libCellName])){
            it.second->fromBlock->area -= techs[it.second->fromBlock->techs]
[it.second->libCellName];
            it.second->toBlock->area += techs[it.second->toBlock->techs]
[it.second->libCellName];
            swap(it.second->fromBlock, it.second->toBlock);
        }
    }
}
```

Result

As a comparison, I also implement the initial partitioning by put the cells into the die A first, and then put the cells into the die B. The result is not as good as the formula above.

Testcase	Cut Size before optimization	Cut Size after optimization	Runtime before optimization	Runtime after optimization
public1	208	193	0.03	0.03
public2	3661	3666	1.7	1.63
public3	6598	7092	163.03	50.23
public4	2248	2265	1.11	1.19
public5	2127	1669	19.55	15.77
public6	fail	10281	TLE	233.59

As you can see, the cut size is almost the same. However, the runtime is improved a lot.

Boost C++ Library

Before I use boost C++ library, the runtime of the program is not satisfying on testcase public6.

1. Since lots of the data structure is implemented by unordered_map, which is a hash table.
2. In theoretically, the time complexity of unordered_map is O(1). However, the hash function of unordered_map is not perfect. So, the time complexity of unordered_map is not always O(1).
3. I tried to use vector instead of unordered_map, but the program have a lot of searching and inserting. And the structure of my program is not suitable for vector. So, I decided to use boost C++ library.
4. After using boost C++ library, the hash table is replaced by boost::unordered_map. The runtime of the program is improved a lot.

Furthermore, I use boost::map to store the buckets. Just want to try whether it will improve the runtime or not. However, the result is almost the same as using std::map.

Result

Testcase	Cut Size before optimization	Cut Size after optimization	Runtime before optimization	Runtime after optimization
public1	195	193	0.10	0.03
public2	3680	3666	5.63	1.63
public3	6890	7092	195.12	50.23
public4	2036	2265	3.01	1.19
public5	1649	1669	45.32	15.77
public6	fail	10281	TLE	233.59

As you can see, the cut size is almost the same. However, the runtime is improved a lot.

What have you learned from this homework? What problem(s) have you encountered in this homework?

1. The data structure is very important. It will affect the runtime of the program.
2. The problem is very complicated. I had a hard time to understand the algorithm and implement it. Besides, In every function, I have to consider the data structure that I use. It's very hard to implement.
3. I learned how to use boost C++ library. It's very useful and powerful. I will use it in the future.
4. Debugging is a big problem. I have to check every function and every line of the code. Otherwise, I will get a wrong result. and print out the buckets content is very unrealistic. Since content in the buckets is a lot, and the content is unordered. So, I have to check the code flow and the result of the program to debug.